Spring 2026: Projects in Usable Security: Design Document Template

Umme, Helen, Maria, Jada

June 15, 2026

Contents

1 Executive Summary

From student flyers to payment methods to advertisements, QR codes have only become more popular in recent years. SafeScan is an application that can be used to scan these QR codes to help assess the safety of the embedded links. SafeScan takes a different approach from other QR code scanner applications like DNSChecker, QsecR or the sIQuirity Foundation QR Code Checker because it combines the Google Safe Browsing API with various heuristics, returning a risk score and reason. The user can use this information to aid their judgment on whether they want to navigate to the destination url of the QR code.

The goals of SafeScan are to allow a user to scan a QR code with the camera, assess the safety of the QR code, and allow the user to go directly to the destination. SafeScan is secure because it prevents direct navigation to unfamiliar urls, validates user input, has accurate multi-level threat detection, and minimizes the exposure of user data to external services and libraries. SafeScan is not meant to allow users to upload their own pictures, integrated with their camera app, or provide users with tips about phishing or other malware avoidance techniques. We also do not guarantee detection of all malicious urls, nor do we provide device-wide malware protection or full sender identity verification.

Assuming SafeScan works as intended and is without vulnerabilities, it provides protection for impulsive or unaware users and makes security accessible for users with low technical literacy. Those who already employ advanced security tools, who do not scan QR codes, or do not have smartphones will likely not benefit much from SafeScan. Nevertheless, there are still some harms associated with SafeScan: users might fall into habits of over reliance, or user data might be logged, analyzed, or otherwise leaked by the third party libraries that SafeScan relies on. If vulnerabilities exist in SafeScan, users might be harmed as they may visit malicious sites that they otherwise would not, their personal data might be exposed, leading to surveillance, profiling, or tracking, and it may cause a systematic erosion of trust in security technology.

SafeScan is overall an ethical software: it respects user autonomy, allowing users to visit sites even if they are marked as unsafe, maximizes benefits while minimizing harm, and supports public interest and maintains trust. However, there is a concern that the benefits might not be distributed fairly, as some users might not benefit from it. SafeScan also follows the following ethical standards: the NIST SP 800 3b digital Identity Guidelines and the OWASP Mobile Application Security Verification Standard.

The project code is published in a Github Repository. Along with the published a code, anyone trying to replicate the project will also need to install Flutter and create a Google Safe Browsing API key. SafeScan is implemented as a Flutter application and uses Dart as the main programming language. The project has 5 main components: QR code scanning, url expansion, Google Safe Browsing, heuristics + scoring, and history. The QR code scanning accesses, the user's camera and decodes a url from a QR code using the mobile_scanner library; if the decoded url is shortened, then it is expanded. After retrieving the full url from a given QR code, it is parsed through the Google Safe Browsing API, specifically the UpdateAPI and LookupAPI (these are subsets of the SafeBrowsingAPI), to check if the given url matches any malicious ones in the database; the results of this are binary. If the API doesn't flag the url as unsafe, then the url is tested through a couple heuristics, including the Levenshtein distance or phishing key words. Lastly, any links tested via scanning can be accessed from the history function, where values are stored in a local file. Even with implementing multiple security features, such as no automatic navigation, url input validation, registrable domain extraction and a heuristics based scoring model, there are still some vulnerabilities that exist within the project. Some of these issues are that SafeScan may not be able to recognize all redirect urls or may confuse url-encoded substrings. There may also be hidden base64 encoded malware within trustworthy sites.

Through multiple cognitive walk-throughs with a subset of the target demographic for SafeScan, we recognized a couple of changes that could be made to the application in order to make the user experience better. Some of the changes that we made as a result of the user studies are adding a search bar to the history feature, and redirecting to the camera page after clicking "Scan Another" instead of the home page.

A security evaluation performed by the team revealed one area of possible weakness in our code. To implement our history function, the sites the user visits are written in plaintext to disk, leaving them vulnerable to any attackers who get hold of the user's device. There also might exist vulnerabilities in the third party libraries we use, such as the SafeBrowsingAPI and the Mobile_Scanner library, which we will not address as these are out of the scope of our project. Peer security audits revealed that our code was mostly secure, with the exception of appending the API key to the request URL, rather than the request header.

2 Project Overview

It is not safe to scan random QR codes because they may be malicious. Our application allows users to scan a QR code and determine if it is malicious or not without the user having to navigate to the URL destination. SafeScan has been implemented as a mobile application through Flutter. The application uses the mobile_scanner library, which allows the application to scan a QR code and decode its url. Once the url is decoded, we validate the input to ensure it is a link and not malformed, as well as check for url squatting. After checking for these things, the decoded url is sent to the Google Safe Browsing API, which is connected to a database of billions of websites online and information on whether they are safe or not. After sending a link to the Google Safe Browsing API, the API tells you if the link is malicious or not. If the link is not flagged by the API, we pass the link through the heuristics to generate a score of how unsafe a link may be. After checking the safety of the link, the application will indicate the safety of the link to the user with a safety score, where 0-34 is safe, 35-69 is suspicious, and 70+ is dangerous. On the page that indicates the safety of the website to the user, the user will be able to navigate directly to the link, view the link, and copy the link to the keyboard. There is also a history feature where the user can see their previously scanned qr codes, and delete any entries from the history. The user can also search through the history with a search bar. Alternatively, from the home page, the user can paste a link directly to assess its safety.

2.1 Motivation and Use Cases

This project is important because nowadays QR codes are very popular and the most basic way to scan them are through the normal camera app. However, the camera app doesn't indicate if the QR code is malicious or not. This project would allow users to assess the safety of a QR code and navigate to the URL safely.

- **Use Case 1: Student Flyers**
  College campuses often have many flyers on display, whether they are for a club event or asking for participation in a research study. There may be students who would be interested in inquiring further, but refrain from scanning the QR code due to safety concerns, or students who scan QR codes without thinking about safety. This application would help the student ensure the safety of the QR code destination and allow them to further explore what is being advertised.
  There are also students that may scan qr codes with no discretion because they are on a college campus; this application can help them ensure that they are making more mindful decisions before scanning.
- **Use Case 2: Payment Method**
  As online payments become more widespread, businesses have started to allow you to pay through a QR code. However, it is possible that customers may be weary of the integrity of where the QR code leads, but also don't carry cash. This application would allow them to confirm that the QR code will lead to a legitimate payment method so that they can send their money safely.
- **Use Case 3: Advertisements**
  In the modern day QR codes have started to be implemented in advertisements. This application would allow the average consumer to confirm that the QR case leads to the expected destination before going to a potentially malicious link.

2.2 Related Works

- **Related System 1: DNSChecker**
  The website DNSChecker is a web based QR code scanner that allows users to upload images or scan QR codes [1]. It then decodes the QR code and returns whether the code contains a URL along with the link and data type. This is similar to our project in that it takes an input of a QR image and decodes the link, allowing users to inspect the QR code without immediately navigating to the destination. However, this service does not directly assess the safety of the decoded URL. The website has a small informational section at the bottom: "Can a QR code be hacked?", relying on the user to read this and interpret the link using their own judgment. Through SafeScan, we instead aim to *explicitly* evaluate the safety of the destination and present this through a simplified, color coded interface, making the security decision quick and understandable.

- **Related System 2: QsecR**
  A research paper proposes and evaluates QsecR [2], which is an Android QR code scanner that detects malicious URLs. This is related to our project because it is also a QR code that assesses the safety of links. QsecR is different from our application because it checks the safety of links with machine learning and a curated dataset, whereas we will use the Google Safe Browsing API and its given database. We have decided not to use machine learning (the basis of QsecR) because implementing machine learning will be time, money, and computationally expensive. The data collection and annotation will take a lot of time and allow us to implement less features.

- **Related System 3: sIQuirity Foundation QR Code Checker**
  The sIQuirity foundation is a cybersecurity nonprofit that provides affordable security services and community programs for those in need. One of their tools is a qr code tester. [3] where you can either scan with your camera or upload photos of qr codes to check for malicious links. This is related to our project because it also scans QR codes and assesses whether they are malicious, but our project uses the Google Safe Browsing API, while this tool did not appear to.

- **Related System 4: Google Safe Browsing**
  Google Safe Browsing is a security service that identifies unsafe websites, including phishing and malware-hosting URLs, and is widely used by modern web browsers. This system is directly related to our project because SafeScan uses the Google Safe Browsing API to assess the safety of QR code destinations. However, Google Safe Browsing operates primarily at the browser level and does not provide a dedicated, user-facing tool for scanning QR codes or previewing destinations before navigation. Our project builds on this service by applying it specifically to QR codes and presenting the results in a simple, mobile-friendly interface.

3 Requirements

3.1 Brief System Summary

The user will open the application on their phone. The application will connect to the phone's camera in order to scan a QR code. The data received from the camera (if it is a valid QR code) will be decoded into a plaintext url, and cross referenced with The Google Safe Browsing API. The API will determine whether or not the QR code destination is safe and send that information back to the application. If the API does not flag the url, then the url will be tested through the heuristics. The application then indicates the safety of the url from the QR code via a risk score. The page that displays the score will also display the url from the QR code and allows users to navigate to the link and/or copy it. From the home page, the user can paste a link directly to assess its safety. There is also a history feature where the user can see all the links they have scanned in the past; there is also a search bar in the history.

3.2 Functionality Goals

- **Functionality Goal 1: Scan QR Code**
  The application allows you to use your camera to access a link from a QR code.

- **Functionality Goal 2: Assess Safety of QR Code Destination**
  The application will check the integrity of the QR code destination by first determining if it is an URL or an URI, and cross referencing the Google Safe Browsing API or the whitelist respectively, then performing heuristic-based checks.

- **Functionality Goal 3: Indicate Safety to User**
  After assessing the QR code, the application will tell the user about the safety of the app with a risk score.

- **Functionality Goal 4: Display url destination from QR code**
  On the same page that indicates the safety of the app, the application will display the link that the

QR code leads to so that the user can use their own discretion on whether to trust the link. The user can also copy the link.

- **Functionality Goal 5: Go Directly to QR Code Destination**
  On the same page that indicates the safety of the app, the user will be able to navigate to the URL from the application.

The functionality goals are listed in order of the user experience. They break down the logistical steps that the application will take in order for the user to scan a QR code, assess the safety and and navigate to the given URL.

3.3 Functionality Non-Goals

- **Functionality Non-Goal 1: Do not allow user to upload picture of QR Code.**
  The user has to scan the QR code with their camera, rather than upload a picture that they had previously taken.
- **Functionality Non-Goal 2: Do not provide users with tips about phishing.**
  The user will not receive detailed feedback about how to avoid phishing in the future.
- **Functionality Non-Goal 3: Do not automatically open the app when it detects a QR code taken with the normal camera app.**
  This app is not an add-on to the camera app. The user will not be prompted to verify all QR codes that they take a picture of using their regular camera app. They will have to specifically open this app for it to work. It will not work in the background.

3.4 Threat Model

3.4.1 Actors

Expected Actors

- **End User**
  A smartphone user who scans QR codes to safely access URLs.
- **Mobile Device / Operating System**
  Provides camera access, networking, and application sandboxing.
- **QR Scanner Application**
  The system under design, responsible for scanning QR codes, validating links, and displaying results.
- **Google Safe Browsing API**
  External service used to assess URL reputation and known threats.
- **Destination Website or Service**
  The web page or application referenced by the QR code.

Unexpected / Potentially Adversarial Actors

- **Malicious QR Code Creator**
  Generates QR codes pointing to phishing, malware, or fraudulent destinations.
- **Compromised or Malicious Website**
  Appears legitimate but hosts harmful content.
- **Network Adversary**
  Attempts to intercept or manipulate network traffic.
- **Malicious Third-Party Applications**
  Other apps on the device attempting to interfere with or monitor QR scanning.
- **Adaptive Scammer**
  An adversary who actively evolves QR-based phishing techniques to evade detection.

3.4.2 Assets

- **User Safety:** Protection from malware, phishing, credential theft, and financial fraud.
- **User Trust:** Confidence that the application provides accurate and reliable safety assessments.
- **Application Integrity:** Correct operation of scanning, validation, and result display logic.
- **API Credentials:** Google Safe Browsing API keys used by the application.
- **User Privacy:** Metadata related to scanned QR codes and user behavior.
- **User Finances:** Protection against fraudulent payments and financial theft.
- **User Personal Information:** Protection of credentials, contact information, and other sensitive data entered into malicious sites.

3.4.3 Security Goals

- **Security Goal 1.** Prevent Automatic Navigation to Malicious URLs. The application must ensure that QR code destinations are never automatically opened without prior safety analysis and explicit user approval.
- **Security Goal 2.** Accurate Threat Detection. The system should reliably detect known malicious, phishing, or deceptive URLs using trusted threat-intelligence sources (e.g., Google Safe Browsing).
- **Security Goal 3.** Protect User Privacy. The application should avoid storing scanned QR codes or URLs unnecessarily and should minimize the exposure of user data to external services.
- **Security Goal 4.** Input Validation and Safe Processing. The system must validate that decoded QR content is properly formatted (e.g., valid URL, reasonable length, expected scheme) before processing or displaying it, in order to prevent malformed input or exploitation.
- **Security Goal 5.** Risk-Level Communication. The system will present a numerical score accompanied by graduated warning levels(e.g., low, medium, high risk) rather than binary safe/unsafe indicators to reduce over-reliance and false confidence.

3.4.4 Security Non-Goals

- **Security Non-goal 1.** Complete Detection of All Malicious URLs. The system does not guarantee detection of all phishing or malicious links, particularly newly registered domains or highly sophisticated attacks, like base-64 encoded malicious HTML or JavaScript that executes automatically.
- **Security Non-goal 2.** User Education on Phishing Techniques. The system does not aim to train users on how to identify phishing or malicious behavior manually.
- **Security Non-goal 3.** Device-Wide Malware Protection. The application is not intended to function as a full antivirus or endpoint protection system.
- **Security Non-goal 4.** Full Sender Identity Verification. The system does not authenticate the identity of QR code creators or website owners. Future enhancements may include local context indicators (e.g., previously scanned domains), but no centralized identity verification is performed.

4 Design

4.1 Description

When opening the app, the user will be presented with a popup asking whether they grant permission for the app to use their phone's camera. This permission is necessary for the app to function, so if they choose to deny, the app will not work. If granted, the user will then be prompted to take a picture of a qr code. Once the app detects a qr code, it converts the qr code from image to a plaintext link. We then send a GET request to the link a maximum of 10 times to find the true final destination of the url (useful in the case of url shorteners). This final url is then passed to the Google SafeBrowsing API, which returns a status of safe or unsafe, encoded as 0 and 1 respectively. If the SafeBrowsingAPI determines that the url is unsafe, then that result is displayed to the user, with a score of 100. If SafeBrowsing returns a 0 corresponding to safe, then the link is passed to our heuristics system. Afterwards, the url is passed to our heuristics system, which first canonicalizes the link to determine if there is any suspicious unicode behaviour, then compares the root of the url to a selected list of top-level-domains to detect typosquatting, and checks for keywords such as "verify" and "login". The heuristic system will add its own score to the output from Google SafeBrowsing and return a total score determining the safety of the link, where a score below 34 means the link is safe, a score between 34 and 69 means that the user should beware, and 70+ represents high risk. The output screen includes the score, the reason for the score, and the link itself, which is a clickable and placed on a background colored according to the score, where low risk/safe would be green, medium risk would be yellow, and high risk would be red. Alternatively, from the home page, the user can paste a link directly to assess its safety. There is also a history feature that shows all the links that have been tested through qr codes. The history feature has a search bar to search through the websites that have been scanned.

4.2 Architectural Diagram

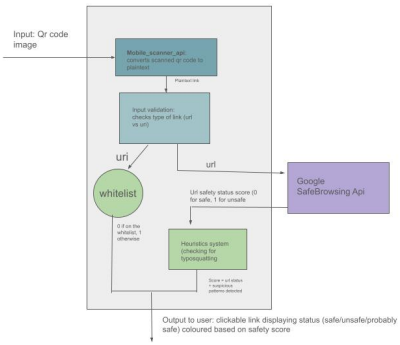

Figure 1: Architectural Diagram of QR Code Safety App

4.3 How Goals Are Met

4.3.1 Functionality

Our functionality goals are met by the app in the following ways:

- **Functionality Goal 1: Scan QR Code**
  The application requests permission to use the camera, which allows it to scan a qr code. The application does not store any pictures.
- **Functionality Goal 2: Assess Safety of QR Code Destination**
  After scanning the QR Code, the application will use the mobile_scanner library to decode it and return a plaintext link. This link will then be evaluated using the the Google Safe Browsing API,

heuristic-based validation, and/or a whitelist. If the application cannot connect to the internet, since we are using the local version of the SafeBrowsingAPI, the app should still function as intended.

- **Functionality Goal 3: Indicate Safety to User**
  Once the app visits the link using either the whitelist or the combination of Google SafeBrowsing and heuristic-based validation, it will return a risk score. A higher risk score means indicates that the link is more likely to be malicious or unsafe.
- **Functionality Goal 4: Preview of QR Code Destination**
  The application will print the link in plaintext with a header to show the first part of the page it leads to.
- **Functionality Goal 5: Go Directly to QR Code Destination**
  The user will be able to click the link and go to the destination.

4.3.2 Security

Our app meets the following security goals:

- **Security Goal 1.** Prevent Automatic Navigation to Malicious URLs. The application only opens links that have been shown to be safe by the Google Safe Browsing API and that have passed the heuristic-based validation tests.
- **Security Goal 2.** Accurate Threat Detection. The system will use the most updated version of Google Safe Browsing API to detect any threats.
- **Security Goal 3.** Protect User Privacy. The application will not store any links visited. Using the UpdateApi(v4), we will download encrypted versions of the Safe Browsing list so the server does not know the actual URLS queried by clients. They will be hashed, compared to the SafeBrowsing list, and processed locally so Google will not know any details. The LookupApi is used as fallback in the case that the UpdateApi is not working.

4.4 Known Security and Other Issues

Security is not a binary, and there will be threats that you cannot protect against. Discuss those known issues here and connect them with your explicit non-goals. Also discuss any other known issues that you do not discuss elsewhere in this document.

You are welcome to introduce subsections for functionality issues and security issues.

For any known vulnerabilities, include a discussion of whether the vulnerabilities are okay (from a risk management perspective) or whether there are ways of mitigating those vulnerabilities in the future.

You will likely update this section throughout the semester, as your own or the peer security analysis uncovers new issues.

4.5 Benefits, Harms, and Ethics Analysis

This section examines the ethical implications and broader societal context of SafeScan. We consider benefits and harms under 2 scenarios: when the system operates without vulnerabilities, and when it operates with vulnerabilities.

4.5.1 Benefits (Assuming No Vulnerabilities)

Given SafeScan operates exactly as designed and implemented, it provides several meaningful benefits across stakeholder groups.

- **Benefit (No Vulnerabilities) 1. Protection for Impulsive/Unaware Scanners** One benefit is that SafeScan reduces exposure to phishing, malware, and scams for users who tend to quickly scan QR codes without thinking about risk. QR codes have become increasingly common as restaurants,

payment systems, etc. turn to digital solutions. One example is college club posters that often appear near campus on bus stations or poles. Since these appear right outside campus, a college student scanning these may be unsuspecting. SafeScan intervenes by identifying the malicious link before the user actually opens it in their browser.

- **Benefit (No Vulnerabilities) 2. Security Accessibility for Low Technical Literacy Users** SafeScan simplifies security, so users do not need to understand the construction of URLs, browser security indicators, or different phishing heuristics. Elderly users, children, or individuals with limited technical familiarity often do not know how to distinguish legitimate domains from malicious ones. By translating technical risk into clear, simple warnings, SafeScan makes security more accessible.
- **Non-beneficiaries** Parties that may not see much benefit include people who already employ advanced security tools (such as anti-phishing browser extensions) and those who do not scan QR codes or do not have phones.
- **Distribution of Benefits and Power** SafeScan benefits those with lower technical literacy the most, providing an easy, straightforward solution. It gives them visibility into potentially unsafe actions.

4.5.2 Harms (Assuming No Vulnerabilities)

Even if SafeScan operates exactly as designed and implemented, there still may be certain harms.

- **Harm (No Vulnerabilities) 1. Overreliance and Reduced Security Literacy** One harm may be that users, especially those with little technical knowledge, may begin to over-rely on automated safety systems. A user that repeatedly sees "safe" labels may stop paying attention to URLs, reducing user agency. This could inadvertently weaken broader security awareness and due diligence while placing more decision making in the hands of technology.
- **Harm (No Vulnerabilities) 2. Centralization of Power and Data** Since SafeScan relies on external threat intelligence services (i.e. Google Safe Browsing, this brings a concern regarding data centralization and surveillance. Though the UpdateAPi ensures that data is processed locally, the LookupAPI is used when the UpdateAPI fails, and in these cases, Google would be an adversary as a powerful intermediary with lots of visibility into the user, as the user's data may be logged or analyzed by them.

4.5.3 Benefits (If One or More Vulnerabilities Exist)

If a vulnerability exists in our system, the beneficiaries would change significantly.

- **Benefit (Vulnerabilities) 1. Exploitation by Attackers** A vulnerability may benefit attackers by allowing them to bypass detection easily under the guise of a safe and trustworthy tool. It would make them appear credible and potentially scam / collect data on users.
- **Non-beneficiaries** Legit users / organizations would no longer fully benefit, as vulnerabilities could undermine the system's reliability.

4.5.4 Harms (If One or More Vulnerabilities Exist)

If a vulnerability exists in our system, the harms may extend to an even broader audience.

- **Harm (Vulnerabilities) 1. Amplified False Sense of Security** All types of users may be harmed by a false sense of security, leading to greater harm. Misplaced trust in an incorrect system would lead to users visiting malicious sites they otherwise may have raised more suspicion towards and avoided.
- **Harm (Vulnerabilities) 2. Privacy and Surveillance Exposure** Any users that scan and are exposed to malicious QR links would be harmed by privacy violations and surveillance risks. Their personal identifiable data may be exposed, enabling surveillance, profiling, or tracking.

- **Harm (Vulnerabilities) 3. Systemic Erosion of Trust in Security Technology** More broadly, this may cause an overall erosion of trust in security tools and public facing technology for affected users.

4.5.5 Framework Analysis

For our ethical analysis, we use The Menlo Report, which provides ethical guidelines for computer security research and technology design. The Menlo Report focuses on four principles: respect for persons, beneficence, justice, and respect for law and public interest. These principles help evaluate whether a security system protects users while also considering privacy, fairness, and potential harm.

- **Question 1: Does SafeScan respect user autonomy?** One question raised by the Menlo Report is whether users are able to make their own informed decisions when interacting with the system. SafeScan is designed to support user autonomy by showing the decoded QR code link and risk level before opening the website. Instead of automatically navigating to the destination, the app pauses and allows the user to decide whether they want to proceed. However, a potential concern is that users may over-rely on the system's judgment. If a link is labeled "safe," users may assume that the link is completely trustworthy and stop thinking critically about what they are opening. To address this concern, the system presents graduated risk levels (low, warning, high) rather than only a binary safe/unsafe label. This communicates uncertainty and encourages users to remain cautious. In the future, the system could also include clearer messaging that explains that the risk assessment is not a guarantee of safety.

- **Question 2: Does SafeScan maximize benefits while minimizing harm?** The Menlo Report's principle of beneficence asks whether a system provides meaningful benefits while minimizing potential harm. SafeScan provides clear benefits by helping users detect malicious QR code destinations before opening them. QR codes hide their destination links, which makes them a common tool in phishing scams. By decoding the URL, validating the input, checking Google Safe Browsing, and applying phishing heuristics, SafeScan helps reduce the likelihood that users accidentally open malicious sites. At the same time, there are possible harms. The system could misclassify links, either flagging legitimate websites as risky or incorrectly labeling malicious sites as safe. Additionally, because the system uses Google Safe Browsing, some URL information may be shared with an external service. To reduce these harms, the system does not store scanned URLs locally and only sends minimal information needed for the reputation check. The risk-level system also avoids making absolute claims about safety. In the future, additional local analysis could reduce reliance on external services.

- **Question 3: Are the benefits distributed fairly?** The Menlo Report's principle of justice asks whether a system benefits different groups of people fairly. SafeScan is intended to help a wide range of smartphone users, but it may be especially helpful for people with lower technical literacy, such as elderly users or individuals who are unfamiliar with phishing techniques. These users may benefit most from simple warnings and clear explanations. However, the system may not benefit all users equally. People who already use advanced security tools may not gain much additional protection from SafeScan. There is also a risk that the heuristic detection system could incorrectly flag certain legitimate URLs if they follow patterns that resemble phishing links. To address this concern, the heuristics system should be tested on a wide variety of legitimate and malicious URLs to reduce false positives. Ensuring that warnings are simple and easy to understand also helps ensure that the users who most need protection can benefit from the system.

- **Question 4: Does SafeScan support the public interest and maintain trust?** Another principle in the Menlo Report is respect for law and the public interest. Security technologies should contribute positively to society and maintain public trust. SafeScan aims to improve everyday digital safety by helping users avoid QR-code-based scams that appear on posters, advertisements, and public spaces. However, if the system fails or gives incorrect safety assessments, users could develop a false sense of security or lose trust in security tools more generally. To address this concern, the system clearly states its security non-goals, such as not guaranteeing the detection of every malicious URL and not functioning as a full antivirus system. Being transparent about the system's limitations

helps maintain trust and prevents users from assuming that the tool provides perfect protection. To communicate this to the user, we will have a disclaimer on the home screen of the app, stating that the system is not foolproof, and there is the possibility that some malicious urls are not detected.

4.6 Engineering Standards and Best Practices

Please list **at least one** engineering or professional standard (or best practice) that you needed to understand/leverage in your design and implementation, or that a commercial design/implementation/deployment of your project would need to meet. For example, if your project works with RFID, you might need to rely on RFID ISO standards. As another example, if your project uses cryptography, your choices should meet the relevant NIST cryptography standards. As another example, if your project deals with passwords, you should consult the NIST Password Guidelines. Depending on your project, these or other standards might be relevant.

For each standard, please **explain** how it is relevant to your project, and how the design **incorporates** or **meets** this standard. (If the design deviates from the standard, or if there are no relevant standards, please explain this thoughtfully instead.)

- **NIST SP 800-63B Digital Identity Guidelines** : NIST SP 800-63B Digital Identity Guidelines emphasize phishing resistance, secure user interaction, and privacy first system design. SafeScan follows these principles by preventing automatic navigation to QR code destinations and requiring explicit user action to open links. Instead of presenting absolute "safe" guarantees, SafeScan uses graduated risk levels to communicate uncertainty and reduce over-reliance on automated security decisions.
- **OWASP Mobile Application Security Verification Standard (MASVS:** OWASP MASVS details best practices for secure mobile application development, which SafeScan follows by validating and canonicalizing all decoded QR code URls before processing them. This helps reduce malformed input and parsing vulnerabilities. SafeScan communicates with Google Safe Browsing API over HTTPS, stores API credentials in environment config files rather than hard coding them, and follows the principle of least privilege by only requesting camera permissions necessary for QR scanning.
- **Google Safe Browsing API Best Practices:** SafeScan follows Google Safe Browsing API's best practices by validating and canonicalizing URLs before analysis, as well as minimizing unnecessary exposure of user browsing behavior. SafeScan uses Google Safe Browsing as a threat-checking source in combination with independent heuristic based phishing checks, creating a layered, resilient defense approach.

5 Implementation

You will likely use one or more subsections to structure the description of your system's implementation.

5.1 Running Your Project

- The **GitHub** repository for the project is linked here: `https://github.com/Umme-R/safe_scan_flutter`
  - This repository stores the Flutter App, which contains Dart code for both an Android and iOS mobile application.
- It is necessary to install the **Flutter Extension** into **VSCode**: `https://docs.flutter.dev/install/quick`
- Install NodeJS by following the instructions linked here: `https://nodejs.org/en/download`
- SafeScan uses the Google Safe Browsing API and the API key is hidden for safety reasons. In order to recreate this project, it is necessary to create your own instance of a Google Safe Browsing API key and add it to a .env file within the project. The API key should specifically be set in the following file path: assets/config/app.env.

  - https://developers.google.com/safe-browsing/v4/get-started

- In order to test either the Android or iOS application, it must be run on either a simulator/emulator or a physical device. For sake of ease in testing, it is also possible to view the application in a browser tab on you laptop.
  - **iOS:** The simulator to test for iOS is **Xcode**, and requires a MacBook for testing
    - https://docs.flutter.dev/platform-integration/ios/setup
  - **Android:** To test the app on android, after cloning the repository, open a terminal window in the cloned folder. Run flutter pub get, then flutter build apk. The resulting apk will be in the build/apps/outputs/flutter-apk folder. Send this apk to your android device, then install it.
  - **Web:** Open a terminal window in the cloned folder from the repository. Run the following command:

    `node server.js`

    Next, open another terminal window in the cloned folder and run the following commands. These commands may take a few minutes to run.

    `flutter pub get`

    `flutter run -d chrome`

    This should open a chrome window to run the app in debug mode.

5.2 Application Implementation

Our app was implemented almost entirely in dart, with one localhost server written in JavaScript to allow for url expansion. The main components of our app (roughly in order of execution) are the following:

5.2.1 QR Code Scanning

When the user clicks the "Scan QR Code" button, they are taken to the QRCodeScanner route. Flutter applications are route-based, where every route represents a page. The implementation for the qr code scanning is in the `qr_code_scanner.dart` file. The functionality for scanning QR codes relies heavily on the Mobile Scanner package, which detects barcodes and qr codes. A MobileScannerController object is created, which controls the camera and stops it. Once a QR code is detected within the camera's sights, the raw value of the url is stored in a variable, the controller stops, and the url is sent to the url expander.

5.2.2 Url Expansion

In order to expand potential shortlinks, such as those created by url shorteners like bit.ly, the url from the scanned QR code is sent to the link expansion function found in `qr_code_scanner.dart`. This function relies on `server.js`, which runs a local proxy server at localhost:3000. This proxy server is needed because on Flutter web, the browser prohibits flutter from following redirects from shortened urls as they do not have the required CORS headers. The expandUrl function sends a HTTP get request to the proxy server with the url appended to the end. After a maximum of ten redirects, it takes the body of the client response, parses the json, and returns the expandedUrl attribute of the response, which corresponds to the final destination of the shortened url. If this function does not work, then the function returns the original url.

5.2.3 Google Safe Browsing Integration

The expanded url is then sent through the Google Safe Browsing API. Code for this is found in the `safe_browsing_service.dart` file. Here, we implement both the UpdateAPI (more secure), and the LookupAPI (the UpdateAPI can be finicky, even with a properly configured API key). The threat lists from Google are stored in a local database, updated every 30 minutes. We compute the SHA-256 hash for the url, and use the first four bytes of the hash to check the local threat lists for potential matches. If there are no matches, then the url is deemed to safe by the SafeBrowsingAPI, and goes on to our heuristic check. Our LookupAPI implementation sends the url in a HTTP request to the SafeBrowsingAPI, which returns a value of safe or unsafe.

5.2.4 Heuristics and Scoring

Our heuristics are implemented in `url_heuristics.dart`. Here, we have curated a list of trusted domains (this list does not contain all whitelisted domains in existence as that would be infeasible; rather, we chose a few commonly spoofed ones to test for typosquatting), suspicious top level domains, and phishing keywords. If a url is marked as safe by SafeBrowsing, we first extract the host, full url, and root domain, and calculate the Levenshtein distance between the root domain and each url in the trusted domain to detect typosquatting. We then check if the host ends with any of the suspicious top level domains, whether the host is an IP address (as opposed to a web address), and whether the domain has too many hyphens, subdomains, or contains any of the phishing keywords. For each of these offenses, points are added to the score, which is capped at 100. The score determines the verdict (whether dangerous, suspicious, or safe), which is returned, along with the reason (multiple / major / minor risk factors detected), and the flags, which correspond to the specific offenses the url has committed. This result is then displayed on the scan result screen.

5.2.5 History

Once the result is returned, it is saved to the history. the `history_store.dart` file contains the details for this implementation. The url, display name, safety status, and time of scanning are saved to a local file. The contents of this file persist across sessions, even after the app is closed.

5.3 Implementation Security

Discuss the security of your implementation here. As we will discuss in class, the security of the implementation is different than the security of the design.

5.3.1 Achieved Security Properties

Our system achieves most of the security properties we set out to achieve.

- **No Automatic Navigation (enforced at code level)** Our system ensures that decoded URLs are never automatically opened. The navigation button to the URL is only enabled after URL validation ¿ Google Safe Browsing API response ¿ heuristic phishing checks. This ensures compliance with Security Goal 1.
- **URL Input Validation and Canonicalization** After extracting the URL from the QR code, it then goes through a URI parser, which means that all special characters (anything except numbers, letters, and some punctuation) are percent-encoded to legal ASCII characters, ensuring that these do not interfere with the url's interpretation. The resulting URI is then expanded using a GET request, which is used to get to the final destination of url shorteners and link redirectors. Any invalid URLs are flagged, preventing malformed attacks.
- **Registrable Domain Extraction** We use simple string splitting (splitting at every period in the domain and counting) to extract the true registrable domain, preventing subdomain based spoofing and cybersquatting.
- **Heuristic Based Risk Scoring Model** We created a weighted risk scoring model that evaluates typosquatting, cybersquatting, homoglyph attacks, and suspicious keywords. Through weighted risk thresholds, we can determine whether a URL can be labeled safe (0-34), warning (35-69), or high risk (70-100). This prevents against newly registered phishing domains and other urls that the SafeBrowsingAPI didn't catch. There were no existing granularized scoring systems for heuristics that we could find for reference, so we designed our own. One shortcoming of this is that the numbers are arbitrary, but we justified this decision because our heuristics themselves were solid, based on factors that were generally considered to be good indicators of suspicious links. A very simplified version of our heuristics system is below.

```
def score_url(url):
score = 0
```

```
domain = extract_domain(url)
if is_typo(domain): // check for typosquatting
    score += 45
if is_ip_address(url): // check for valid IP address
    score += 30
if has_mixed_scripts(domain): // check for mixed scripts
    score += 2
if len(url) > 150: // check for suspiciously long URLs
    score += 10
if "login" in url or "verify" in url: // check for suspicious keywords
    score += 20
if hyphen_count > 5 or subdomain_count > 3: // check for irregular characters or subdomains
    score += 30
return score
```

- **Google Safe Browsing** Lastly, we are layering these protection practices with Google Safe Browsing API to see if either flags the URL as high risk. This layered detection method strengthens our system overall, and combines a vetted source with independent detection logic. This helps us achieve Security Goal 3.

5.3.2 Known Issues

Some vulnerabilities that exist in our current implementation are the following:

- **Susceptible to covert url redirection/url cloaking:** Some urls contain redirect parameters in them, or are shortened using url shorteners for aesthetics, or to obscure the parameters, such as affiliate links, embedded in urls. Our app should be able to detect obvious redirect exploits, but more covert ones, such as those that hide their behaviour when it believes a bot is checking it, cannot be detected by our app at this time. From a risk management perspective, I think this vulnerability is reasonable, as sites that obscure their redirects in this way appear to be rare. In the future, this could be mitigated by using different / more robust url expanders.
- **Url Confusion Vulnerabilities:** If urls have url-encoded substrings where they are not expected, url parsers may be confused by this behaviour and behave unpredictably. Regex cannot always detect these. Risk management wise, this vulnerability also appears to be acceptable, because in the cases that I found, the url parser redirected to localhost. However, it is unknown whether attackers could tailor their url parsers to redirect to a malicious direction of their choice.
- **Hidden Base64 encoding:** Our app aims to detect and prevent the execution base64 encoded code within a url itself, but if an otherwise trustworthy site had base64 encoded malware embedded in an image tag, our app would not be able to detect that. This vulnerability seems slightly bad, but I cannot think of any ways to mitigate this that are within the scope of our application. Preliminary research shows that thoroughly scanning all elements of a site could be a potential solution.

6 Usability Evaluation

This is a summative section on the usability of your system.

6.1 Peer Evaluation

Our peer evaluation was conducted by Shriya, Janice, Nicole, and Jasper, who tested the application on Chrome and Firefox. Their initial impressions were positive, describing the interface as intuitive, fully responsive, and featuring seamless camera integration.

Laws of UX aligned with:

- **Hick's Law**: SafeScan "minimizes the number of decisions on each screen," with each screen focused on a single dominant action.
- **Fitts's Law**: Buttons were described as "large, visible, and easy to tap."
- **Jakob's Law**: SafeScan mirrors "familiar interaction patterns users already know," including a centered landing page, a framing-box camera interface, and a results page with a warning icon.

Laws of UX not aligned with:

- **Tesler's Law**: Simultaneously labeling a scan as "Dangerous" while showing "Error: Invalid URL" pushes decision-making complexity onto the user.
- **Miller's Law**: Displaying a very long destination URL string overloads working memory.
- **Postel's Law**: Mixed signals between "Dangerous," "Invalid URL," and an "Open Anyway" button left it unclear whether the issue was a scanning error or an actual threat.

Recommendations: The evaluators suggested replacing technical error labels with plain-language explanations, truncating the displayed URL to show only the domain with an optional full-detail expansion, and distinguishing clearly between error cases such as "Malformed QR code" and "Potentially malicious link."

Response: We plan to revise the results screen to show brief status messages in clear and easy to read language that distinguish between malformed and malicious links, and to truncate displayed URLs to the domain with an expandable option. These changes align with feedback from our own cognitive walkthroughs as well.

6.2 Cognitive Walkthrough

6.2.1 Set-up

We set up the application on a mobile device using a simulator. This would allow users to test the application in the way they would in the real world. We chose a couple of qr codes in advance, some of which were malicious. The malicious links were taken from a phishing link database found on GitHub, and were chosen because they were benign enough that if participants clicked on them, there would be no serious harm done to the user or their device.
The goals we asked them to complete (or examined to see if they naturally did on their own were the following:

- Scan a safe qr code
- Scan unsafe qr code
- Assess the safety of qr code (after scanning code, see if they can interpret the results).
- Navigate to url after scanning qr code
- Navigate to history tab and find/delete options

We picked these goals because we feel that it ensures that the user can see the scope of the app with different situations.

6.2.2 Cognitive Walk-Through Results

User: Gaby, Interviewers: Maria/Helen
Gaby found it very easy to follow through the entire process. She noted it that it was intuitive to use and didn't struggle to move through each step. Gaby found it hard to understand what criteria was being used to flag something as safe or not. She was stuck on what to do after seeing that a link was flagged as dangerous. She thought she understood the bottom bar of buttons (phishing, malware, threats), but expected more substantial / useful information when she clicked on them. Regarding bugs, she found that some links that

seemed safe were marked as unsafe, leading to user confusion.
Overall, Gaby thinks that the idea overall is very useful. She suggests the option of Safe Scan being something that could be installed, that links could pass through automatically with out having to manually input them. She would definitely use it as long as it remains free.

User: Mina, Interviewers: Maria/Helen
Mina found the system very easy to use overall. She immediately understood where to scan the QR code and where to copy and paste links, and described the interface as very intuitive. She noted that the icons were self-explanatory and did not find anything particularly difficult or confusing while navigating the product. She did not encounter any bugs during her use.
Mina really liked how quickly the scanner works, especially compared to scanning QR codes on a phone, which she noted can take longer to register. She appreciated the color-coded system, saying that the green and red indicators made it very clear whether something was safe or unsafe, and found it easy to understand where to click and how to proceed. She also liked the overall UI. While she mentioned that she does not frequently scan QR codes and does not usually think about her phone's security, she said she might still use the product because of how fast and convenient the scanning feature is. She was also able to clearly understand the connection to security, recognizing that there are many unsafe links and malicious QR codes.

User: Advita, Interviewers: Jada/Umme
During the walkthrough, Advita found the process of scanning qr codes intuitive. She mentioned that it felt very quick, so much so that she doubted that the system was actually scanning the QR codes being displayed. After scanning a qr code and hitting the "scan another" button, she was surprised that the app returned to the home screen, rather than the camera screen. The history function was also usable, though she mentioned that she knew what the icon meant but was unsure whether other users would also be aware. She was dissatisfied that the history function did not allow the user to click on the links, and mentioned wanting a search feature. One bug that was encountered was that the app marked a url as safe while the browser flagged it as being a phishing link. Afterwards, Advita rated the system as overall being usable.

User: Mayra, Interviewers: Jada/Umme
During the walkthrough, Mayra was easily navigating through the app. There weren't any points of confusion. There weren't any steps that she misinterpreted. She was surprised that we had an "Open Anyways" button for links that were deemed unsafe, and we explained to her that it is there if the user uses their own discretion to follow the link. She also stated that she probably would not use the copy link button, and it seemed unnecessary with an existing button to navigate to the link. The history function was also easy to use, but she recommended a filter for the history to find a specific website, or to show only safe or unsafe. She also recommended that we have a confirmation before a user can delete all the items in history. She also wants to be able to open links from the history. Afterwards she mentioned that the code scanned very quickly.

6.2.3 System Usability Scale results

Users said that they would like to use the system frequently, found it easy to use (and not unnecessarily complex), deemed the functions to be well-integrated, and believed that most people would learn to use the system quickly, as there were not a lot of things to learn before getting going. They felt confident using the system, as there was not too much inconsistency, did not find it cumbersome, and did not think that they would need a technical person to be able to use the service.

Mina: 5,1,5,1,4,1,5,1,5,1
Mayra: 4,1,5,1,5,1,5,1,5,1
Advita: 4,1,5,1,4,1,4,1,4,1

6.2.4 Take-aways

We learned that some features we added for safety, such as the copy link button, wouldn't be used by the average user. We may just have a box to display the link for the user's discretion, and remove the copy link button. We will also allow the user to navigate to a website from the history, instead of copying the link. We also plan to add a filtering system to the history tab. We will also make the scan another button go to camera directly. We also plan to make the outline of the camera larger on the screen to make users feel safer. Overall, we plan to keep the flow of the application the same.

7 Security Evaluation

This is a summative section on the security of your system.

7.1 Security Evaluation from the Team

As part of this course, we will ask you to conduct a security evaluation of your own project. Please fill out the following sections.

7.1.1 Areas of Possible Weakness

- **History Storage Vulnerability:** For the history function to work, the program stores the urls visited, their display name, their safety status, and their time visited as a JSON-encoded string, which is then written to disk. This data is not encrypted, so one weakness of this particular approach could be that a bad actor could get a hold of this data if they got the user's device.

7.1.2 Areas Out of Scope

This application connects to the Google Safe Browsing API, which may have its own bugs or associated vulnerabilities. This application also utilizes the mobile_scanner library to scan QR codes and decode them into links, and any vulnerabilities here, which could result in user data being compromised, is out of our scope as developers.

7.1.3 Within-team Peer Audits

Each component of the system should receive a within-team peer review (source code audit). Report on that peer review here.

Groups may divide labor between group members, with group members working on different pieces of the software. For such groups, team members should review the parts of the system that they did not write. For example, if you have three team members, and team member 1 wrote the code for some component, then team members 2 and 3 will review that component and report on the results of that review here.

If your group pair programs everything, then review and report on the security of each component of your system, as a team.

You should report on (a) what methodology / process you used to evaluate your software, (b) what things you looked at that you concluded to be sufficiently secure, and (c) what things you looked at that might have security issues. For each issue that you identify in (c), state whether or not you plan to address the issue in the final implementation.

URL-expansion:
(a) Used a CLI tool to navigate where the associated code is and then manually looked through the code to see how it works and evaluate
(b) There is a maxRedirect variable that ensures that we will not have any infinite loops. The flow of the url expansion is also simple to ensure that unwanted results do not show up and mess up the link that is being sent to the url. (c) The application is currently coded to run on local host, so if we use it on a phone, it may not have the same safety features as our the local machines. Based on if we actually publish the app, we may update this implementation.

Scanning: (a) We reviewed all Dart files related to scanning, including the scanner, scan result screen, Safe Browsing service, and history storage files. (b) QR detection uses the open source mobile_scanner library, which has already been publicly reviewed and tested by developers. To prevent bugs or repeated scans, the app uses checks like scan speed limits, processing flags, and stopping the camera after a successful scan. We also use the QR's raw text value directly instead of automatically parsing it, which helps reduce security risks from malformed QR codes. Additionally, camera permission is requested when the user opens the scanning page. (c) Camera permissions depend on the device and OS. On some locked down devices such as certain Android phones, permission requests may fail silently - however, we did not encounter this issue during testing, so we do not currently plan to address it further.

SafeBrowsing:

(a) To evaluate the software, I looked through the flutter code that called the SafeBrowsing API and referenced the SafeBrowsing documentation.

(b) Several aspects of the implementation appear secure:

- The API ket is loaded from a `.env` file using `flutter_dotenv` rather than being hardcoded, and the code explicitly returns an error if the key is missing or empty.
- All requests to Google Safe browsing API are made over HTTPS
- API responses are carefully validated: the code checks the HTTP status code, handles empty response bodies, and verifies that the decoded response is the expected type before processing it.
- The JSON parsing logic uses safe type checks (e.g., `whereType<Map<String, dynamic>>()` and null-aware operators) to prevent crashes from unexpected response structures.
- Errors are caught with a try/catch block and returned as a `SafeBrowsingResult` with an error message, rather than crashing the application.

(c) one potential security issue is that the API key is appended directly as a query parameter in the request URL. This means that the key could be exposed in server logs, network monitoring tools, or error messages. A more secure approach would be to pass the API key in the request header instead, but we will not address that in our final implementation.

Acknowledgements

We thank Gaby, Mina, Advita, and Mayra for being willing participants in our usability evaluation. We would also like to thank the members of BEAM (Shriya Mani, Jasper Shen, Nicole Ineza and Janice Wong) for performing a peer usability analysis on our application–your feedback was invaluable in improving our app.

References

[1] DNSChecker. Qr code scanner. `https://dnschecker.org/qr-code-scanner.php`, 2026.

[2] Ahmad Sahban Rafsanjani, Norshaliza Binti Kamaruddin, Hazlifah Mohd Rusli, and Mohammad Dabbagh. Qsecr: Secure qr code scanner according to a novel malicious url detection framework. *IEEE Access*, 11:92523–92539, 2023.

[3] sIQuirity Foundation. Qr code security checker. `https://siqurity.org/tools/qr-tester`, 2026.